

# UNIVERSIDAD NACIONAL DEL ALTIPLANO DE PUNO

## FACULTAD DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA

---

**Docente:** Ing. Fred Torres Cruz

**Estudiante:** Gilma Coaquira Salluca

**Actividad:** N° 001 - Unidad 2

**Tema:** Análisis de Complejidad Computacional y Manejo de Archivos

---

## PARTE 1: EJERCICIOS DE COMPLEJIDAD COMPUTACIONAL

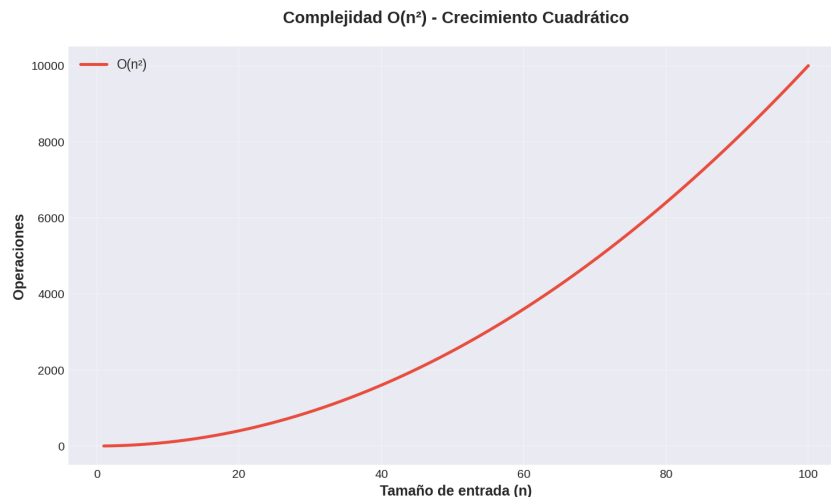
### Ejercicio 1: Complejidad $O(n^2)$

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  double ejecutar_suma_matriz(int n) {
6      int **arr;
7      int suma = 0;
8      long long ops = 0;
9
10     arr = (int**)malloc(n * sizeof(int*));
11     for (int i = 0; i < n; i++) {
12         arr[i] = (int*)malloc(n * sizeof(int));
13         for (int j = 0; j < n; j++) {
14             arr[i][j] = i + j;
15         }
16     }
17
18     clock_t inicio = clock();
19     for (int i = 0; i < n; i++) {
20         for (int j = 0; j < n; j++) {
21             suma += arr[i][j];
22             ops++;
23         }
24     }
25     clock_t fin = clock();
26
27     double tiempo = ((double)(fin - inicio)) / CLOCKS_PER_SEC;
28
29     printf("n=%d, Ops=%lld, Suma=%d, Tiempo=%.6f seg\n",
30           n, ops, suma, tiempo);
31
32     // Liberar memoria
33     for (int i = 0; i < n; i++) {
34         free(arr[i]);
35     }
36     free(arr);
37
38     return tiempo;
39 }
40
41 int main() {
```

```

42 FILE *archivo;
43 int tamanios[] = {100, 500, 1000, 2000, 3000, 5000};
44 int num_tamanios = 6;
45
46 archivo = fopen("datos_ejercicio1.csv", "w");
47 fprintf(archivo, "n,operaciones,tiempo_segundos\n");
48
49 printf("=== EJERCICIO 1: O(n^2) ===\n\n");
50
51 for (int i = 0; i < num_tamanios; i++) {
52     int n = tamanios[i];
53     double tiempo = ejecutar_suma_matriz(n);
54     long long ops = (long long)n * n;
55     fprintf(archivo, "%d,%lld,%.6f\n", n, ops, tiempo);
56 }
57
58 fclose(archivo);
59
60 return 0;
61 }

```



### Captura de compilación:

```

C:\Users\HP\Desktop\ejercici  X + v
=== EJERCICIO 1: O(n^2) ===

n=100, Ops=10000, Suma=990000, Tiempo=0.000000 seg
n=500, Ops=250000, Suma=124750000, Tiempo=0.001000 seg
n=1000, Ops=1000000, Suma=999000000, Tiempo=0.004000 seg
n=2000, Ops=4000000, Suma=-593934592, Tiempo=0.017000 seg
n=3000, Ops=9000000, Suma=1221196224, Tiempo=0.038000 seg
n=5000, Ops=25000000, Suma=420948416, Tiempo=0.105000 seg

-----
Process exited after 3.755 seconds with return value 0
Presione una tecla para continuar . . .

```

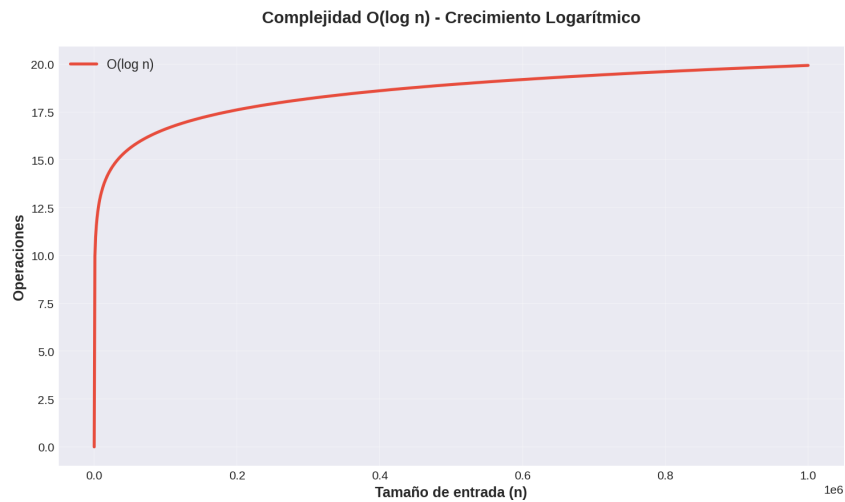
## Ejercicio 2: Complejidad $O(\log n)$ - Búsqueda Binaria

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4 #include <math.h>
5
6 int busqueda_binaria(int arr[], int n, int x, int *comp) {
7     int izq = 0, der = n - 1;
8     *comp = 0;
9     while (izq <= der) {
10         int medio = izq + (der - izq) / 2;
11         (*comp)++;
12         if (arr[medio] == x) return medio;
13         if (arr[medio] < x) izq = medio + 1;
14         else der = medio - 1;
15     }
16     return -1;
17 }
18
19 double ejecutar_busqueda_binaria(int n) {
20     int *arr = (int*)malloc(n * sizeof(int));
21
22     for (int i = 0; i < n; i++) {
23         arr[i] = i * 2;
24     }
25
26     clock_t inicio = clock();
27
28     int iteraciones = 10000;
29     int comparaciones_total = 0;
30     for (int i = 0; i < iteraciones; i++) {
31         int buscar = (i % n) * 2;
32         int comp = 0;
33         busqueda_binaria(arr, n, buscar, &comp);
34         comparaciones_total += comp;
35     }
36
37     clock_t fin = clock();
38     double tiempo = ((double)(fin - inicio)) / CLOCKS_PER_SEC;
39
40     int comp_max = (int)ceil(log2(n));
41     double comp_prom = (double)comparaciones_total / iteraciones;
42
43     printf("n=%d, Comp_max=%d, Comp_prom=%.2f, Tiempo=%.6f seg\n",
44           n, comp_max, comp_prom, tiempo);
45
46     free(arr);
47     return tiempo;
48 }
49
50 int main() {
51     FILE *archivo;
52     int tamanios[] = {1000, 10000, 100000, 500000, 1000000,
53                       5000000, 10000000};
54     int num_tamanios = 7;
55
56     archivo = fopen("datos_ejercicio2.csv", "w");
57     fprintf(archivo, "n,comparaciones_maximas,tiempo_segundos\n");
58
59     printf("=== EJERCICIO 2:  $O(\log n)$  ===\n\n");
60
61     for (int i = 0; i < num_tamanios; i++) {
```

```

62     int n = tamanios[i];
63     double tiempo = ejecutar_busqueda_binaria(n);
64     int comp_max = (int)ceil(log2(n));
65     fprintf(archivo, "%d,%d,%.6f\n", n, comp_max, tiempo);
66 }
67
68 fclose(archivo);
69
70 return 0;
71 }

```



Captura de compilación:

```

C:\Users\HP\Desktop\ejercici x + v
=== EJERCICIO 2: O(log n) ===

n=1000, Comp_max=10, Comp_prom=8.99, Tiempo=0.000000 seg
n=10000, Comp_max=14, Comp_prom=12.36, Tiempo=0.004000 seg
n=100000, Comp_max=17, Comp_prom=15.69, Tiempo=0.002000 seg
n=500000, Comp_max=19, Comp_prom=17.95, Tiempo=0.002000 seg
n=1000000, Comp_max=20, Comp_prom=18.95, Tiempo=0.002000 seg
n=5000000, Comp_max=23, Comp_prom=21.32, Tiempo=0.000000 seg
n=10000000, Comp_max=24, Comp_prom=22.32, Tiempo=0.000000 seg

-----
Process exited after 2.822 seconds with return value 0
Presione una tecla para continuar . . .

```

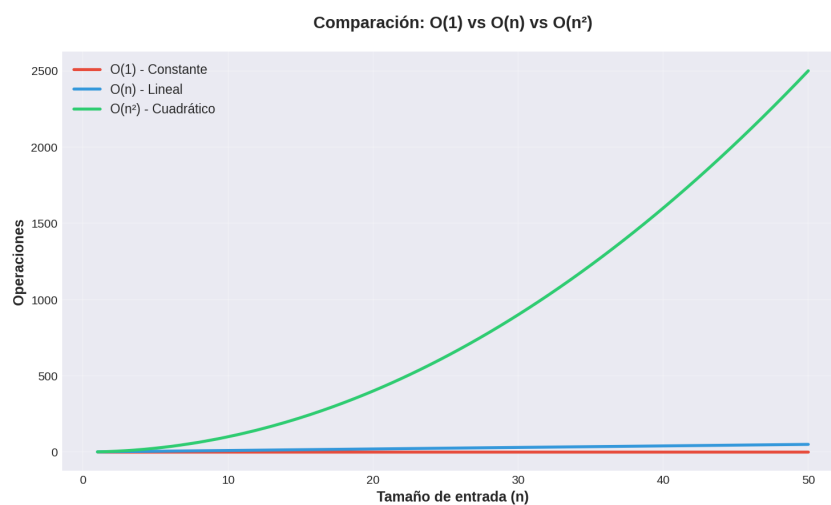
### Ejercicio 3: Comparación $O(1)$ , $O(n)$ , $O(n^2)$

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 typedef struct {
6     int n;
7     double tiempo_a;
8     double tiempo_b;
9     double tiempo_c;
10    long long ops_a;
11    long long ops_b;
12 } Resultado;
13
14 Resultado ejecutar_comparacion(int n) {
15     Resultado res;
16     res.n = n;
17     clock_t inicio, fin;
18
19     //Codigo A: O(n)
20     long long suma_a = 0;
21     res.ops_a = 0;
22     inicio = clock();
23     for (int i = 0; i < n; i++) {
24         suma_a += i;
25         res.ops_a++;
26     }
27     fin = clock();
28     res.tiempo_a = ((double)(fin - inicio)) / CLOCKS_PER_SEC;
29
30     //Codigo B: O(n^2)
31     long long suma_b = 0;
32     res.ops_b = 0;
33     inicio = clock();
34     for (int i = 0; i < n; i++) {
35         for (int j = 0; j < i; j++) {
36             suma_b += j;
37             res.ops_b++;
38         }
39     }
40     fin = clock();
41     res.tiempo_b = ((double)(fin - inicio)) / CLOCKS_PER_SEC;
42
43     //Codigo C: O(1)
44     long long suma_c;
45     inicio = clock();
46     suma_c = (long long)n * (n + 1) / 2;
47     fin = clock();
48     res.tiempo_c = ((double)(fin - inicio)) / CLOCKS_PER_SEC;
49
50     printf("n=%d, A: %.6f seg, B: %.6f seg, C: %.6f seg\n",
51           n, res.tiempo_a, res.tiempo_b, res.tiempo_c);
52
53     return res;
54 }
55
56 int main() {
57     FILE *archivo;
58     int tamanios[] = {1000, 5000, 10000, 20000, 30000};
59     int num_tamanios = 5;
60
61     archivo = fopen("datos_ejercicio3.csv", "w");
```

```

62     fprintf(archivo, "n,tiempo_0n,tiempo_0n2,tiempo_01,ops_n,ops_n2\n");
63
64     printf("=== EJERCICIO 3: Comparacion ===\n\n");
65
66     for (int i = 0; i < num_tamamos; i++) {
67         Resultado r = ejecutar_comparacion(tamamos[i]);
68         fprintf(archivo, "%d,%.6f,%.6f,%.6f,%lld,%lld\n",
69             r.n, r.tiempo_a, r.tiempo_b, r.tiempo_c,
70             r.ops_a, r.ops_b);
71     }
72
73     fclose(archivo);
74
75     return 0;
76 }

```



### Captura de compilación:

```

C:\Users\HP\Desktop\ejercici
=== EJERCICIO 3: Comparacion ===

n=1000, A: 0.000000 seg, B: 0.003000 seg, C: 0.000000 seg
n=5000, A: 0.000000 seg, B: 0.051000 seg, C: 0.000000 seg
n=10000, A: 0.000000 seg, B: 0.212000 seg, C: 0.000000 seg
n=20000, A: 0.000000 seg, B: 0.874000 seg, C: 0.000000 seg
n=30000, A: 0.000000 seg, B: 2.004000 seg, C: 0.000000 seg

-----
Process exited after 8.88 seconds with return value 0
Presione una tecla para continuar . . .

```

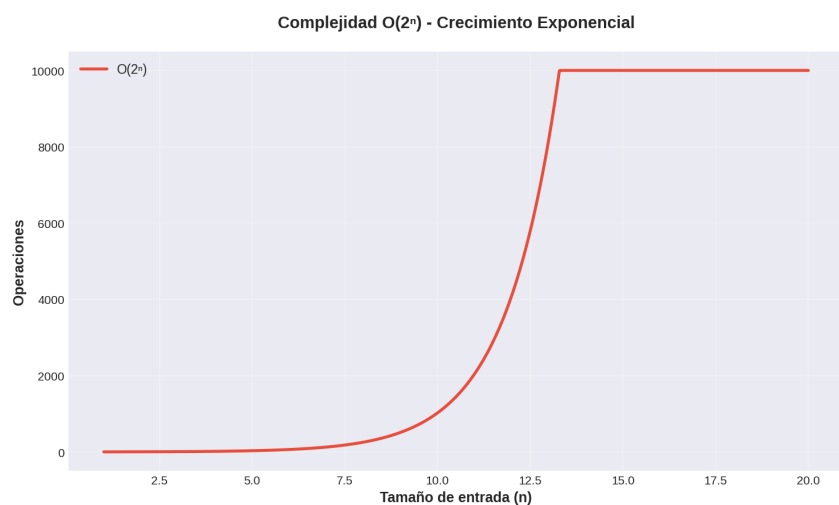
## Ejercicio 4: Complejidad $O(2^n)$ - Fibonacci

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 long long llamadas_recurativas;
6
7 int fib_recurativo(int n) {
8     llamadas_recurativas++;
9     if (n <= 1) return n;
10    return fib_recurativo(n - 1) + fib_recurativo(n - 2);
11 }
12
13 int fib_iterativo(int n) {
14     if (n <= 1) return n;
15     int a = 0, b = 1, c;
16     for (int i = 2; i <= n; i++) {
17         c = a + b;
18         a = b;
19         b = c;
20     }
21     return b;
22 }
23
24 typedef struct {
25     int n;
26     double tiempo_rec;
27     double tiempo_iter;
28     long long llamadas;
29 } ResultadoFib;
30
31 ResultadoFib ejecutar_fibonacci(int n) {
32     ResultadoFib res;
33     res.n = n;
34     clock_t inicio, fin;
35
36     // Recursivo
37     llamadas_recurativas = 0;
38     inicio = clock();
39     int resultado_rec = fib_recurativo(n);
40     fin = clock();
41     res.tiempo_rec = ((double)(fin - inicio)) / CLOCKS_PER_SEC;
42     res.llamadas = llamadas_recurativas;
43
44     // Iterativo
45     inicio = clock();
46     int resultado_iter = fib_iterativo(n);
47     fin = clock();
48     res.tiempo_iter = ((double)(fin - inicio)) / CLOCKS_PER_SEC;
49
50     printf("n=%d, Rec: %.6f seg (%lld llamadas), Iter: %.6f seg\n",
51         n, res.tiempo_rec, res.llamadas, res.tiempo_iter);
52
53     return res;
54 }
55
56 int main() {
57     FILE *archivo;
58     int tamanios[] = {10, 15, 20, 25, 30, 35, 40};
59     int num_tamanios = 7;
60
61     archivo = fopen("datos_ejercicio4.csv", "w");
```

```

62     fprintf(archivo, "n,tiempo_recursoivo,tiempo_iterativo,llamadas\n");
63
64     printf("=== EJERCICIO 4: Fibonacci O(2^n) ===\n\n");
65
66     for (int i = 0; i < num_tamamos; i++) {
67         ResultadoFib r = ejecutar_fibonacci(tamamos[i]);
68         fprintf(archivo, "%d,%.6f,%.6f,%lld\n",
69                 r.n, r.tiempo_rec, r.tiempo_iter, r.llamadas);
70     }
71
72     fclose(archivo);
73     printf("ADVERTENCIA: Valores mayores a 40 pueden tardar mucho\n");
74
75     return 0;
76 }

```



### Captura de compilación:

```

C:\Users\HP\Desktop\ejercici  x  +  v
=== EJERCICIO 4: Fibonacci O(2^n) ===

n=10, Rec: 0.000000 seg (177 llamadas), Iter: 0.000000 seg
n=15, Rec: 0.000000 seg (1973 llamadas), Iter: 0.000000 seg
n=20, Rec: 0.000000 seg (21891 llamadas), Iter: 0.000000 seg
n=25, Rec: 0.000000 seg (242785 llamadas), Iter: 0.000000 seg
n=30, Rec: 0.015000 seg (2692537 llamadas), Iter: 0.000000 seg
n=35, Rec: 0.130000 seg (29860703 llamadas), Iter: 0.000000 seg
n=40, Rec: 1.452000 seg (331160281 llamadas), Iter: 0.000000 seg
ADVERTENCIA: Valores mayores a 40 pueden tardar mucho

-----
Process exited after 6.958 seconds with return value 0
Presione una tecla para continuar . . .

```



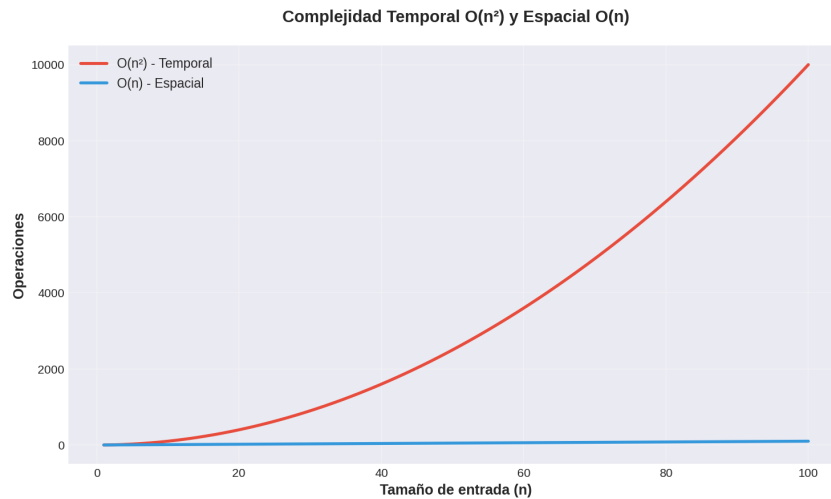
## Ejercicio 5: Complejidad $O(n^2)$ - Eliminar Duplicados

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 int eliminar_duplicados(int arr[], int n, long long *comp) {
6     int *temp = (int*)malloc(n * sizeof(int));
7     int k = 0;
8     *comp = 0;
9
10    for (int i = 0; i < n; i++) {
11        int duplicado = 0;
12        for (int j = 0; j < k; j++) {
13            (*comp)++;
14            if (arr[i] == temp[j]) {
15                duplicado = 1;
16                break;
17            }
18        }
19        if (!duplicado) {
20            temp[k++] = arr[i];
21        }
22    }
23
24    for (int i = 0; i < k; i++) {
25        arr[i] = temp[i];
26    }
27    free(temp);
28    return k;
29 }
30
31 double ejecutar_duplicados(int n) {
32     int *arr = (int*)malloc(n * sizeof(int));
33
34     srand(time(NULL));
35     for (int i = 0; i < n; i++) {
36         arr[i] = rand() % (n / 2);
37     }
38
39     long long comparaciones = 0;
40     clock_t inicio = clock();
41     int unicos = eliminar_duplicados(arr, n, &comparaciones);
42     clock_t fin = clock();
43
44     double tiempo = ((double)(fin - inicio)) / CLOCKS_PER_SEC;
45
46     printf("n=%d, Unicos=%d, Comp=%lld, Tiempo=%.6f seg\n",
47         n, unicos, comparaciones, tiempo);
48
49     free(arr);
50     return tiempo;
51 }
52
53 int main() {
54     FILE *archivo;
55     int tamanios[] = {500, 1000, 2000, 5000, 10000};
56     int num_tamanios = 5;
57
58     archivo = fopen("datos_ejercicio5.csv", "w");
59     fprintf(archivo, "n,tiempo_segundos\n");
60
61     printf("=== EJERCICIO 5: Eliminar Duplicados  $O(n^2)$  ===\n\n");
```

```

62
63     for (int i = 0; i < num_tamamos; i++) {
64         double tiempo = ejecutar_duplicados(tamamos[i]);
65         fprintf(archivo, "%d,%.6f\n", tamamos[i], tiempo);
66     }
67
68     fclose(archivo);
69
70     return 0;
71 }

```



Captura de compilación:

```

C:\Users\HP\Desktop\ejercicio x + v
=== EJERCICIO 5: Eliminar Duplicados O(n^2) ===

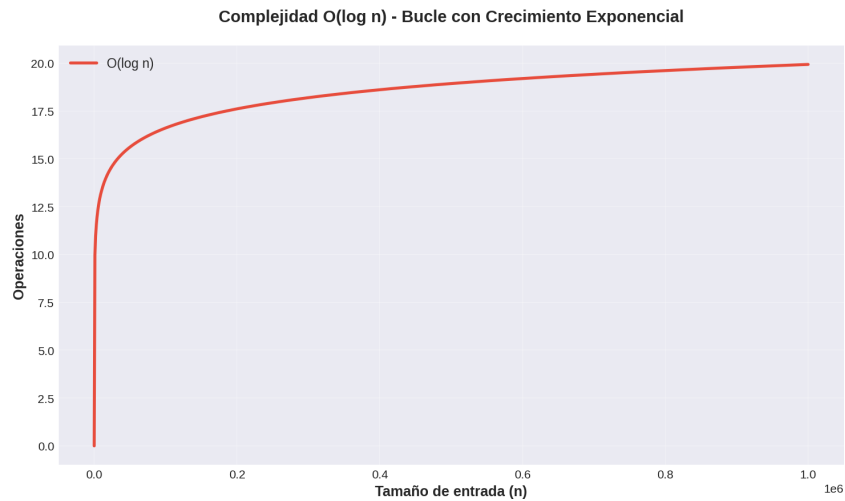
n=500, Unicos=211, Comp=46485, Tiempo=0.000000 seg
n=1000, Unicos=431, Comp=184256, Tiempo=0.001000 seg
n=2000, Unicos=857, Comp=745523, Tiempo=0.003000 seg
n=5000, Unicos=2154, Comp=4676289, Tiempo=0.021000 seg
n=10000, Unicos=4306, Comp=18805838, Tiempo=0.079000 seg

-----
Process exited after 2.791 seconds with return value 0
Presione una tecla para continuar . . .

```

## Ejercicio 6: Complejidad $O(\log n)$ - Bucle Exponencial

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4 #include <math.h>
5
6 int contar_iteraciones(int n) {
7     int count = 0;
8     for (int i = 1; i < n; i = i * 2) {
9         count++;
10    }
11    return count;
12 }
13
14 double ejecutar_bucle_exponencial(int n) {
15     clock_t inicio = clock();
16     int iteraciones = contar_iteraciones(n);
17     clock_t fin = clock();
18
19     double tiempo = ((double)(fin - inicio)) / CLOCKS_PER_SEC;
20     int teorico = (int)ceil(log2(n));
21
22     printf("n=%d, Iteraciones=%d, Teorico=%.0f, Tiempo=%.6f seg\n",
23           n, iteraciones, log2(n), tiempo);
24
25     return tiempo;
26 }
27
28 int main() {
29     FILE *archivo;
30     long long tamanios[] = {1000, 10000, 100000, 1000000,
31                             10000000, 100000000, 1000000000LL};
32     int num_tamanios = 7;
33
34     archivo = fopen("datos_ejercicio6.csv", "w");
35     fprintf(archivo, "n,iteraciones,log2_n,tiempo_segundos\n");
36
37     printf("=== EJERCICIO 6: Bucle Exponencial  $O(\log n)$  ===\n\n");
38
39     for (int i = 0; i < num_tamanios; i++) {
40         long long n = tamanios[i];
41         double tiempo = ejecutar_bucle_exponencial(n);
42         int iters = (int)ceil(log2(n));
43         fprintf(archivo, "%lld,%d,%.2f,%.6f\n",
44               n, iters, log2(n), tiempo);
45     }
46
47     fclose(archivo);
48
49     return 0;
50 }
```



### Captura de compilación:

```
C:\Users\HP\Desktop\ejercici X + v
=== EJERCICIO 6: Bucle Exponencial O(log n) ===

n=1000, Iteraciones=10, Teorico=10, Tiempo=0.000000 seg
n=10000, Iteraciones=14, Teorico=13, Tiempo=0.000000 seg
n=100000, Iteraciones=17, Teorico=17, Tiempo=0.000000 seg
n=1000000, Iteraciones=20, Teorico=20, Tiempo=0.000000 seg
n=10000000, Iteraciones=24, Teorico=23, Tiempo=0.000000 seg
n=100000000, Iteraciones=27, Teorico=27, Tiempo=0.000000 seg
n=1000000000, Iteraciones=30, Teorico=30, Tiempo=0.000000 seg

-----
Process exited after 2.298 seconds with return value 0
Presione una tecla para continuar . . .
```

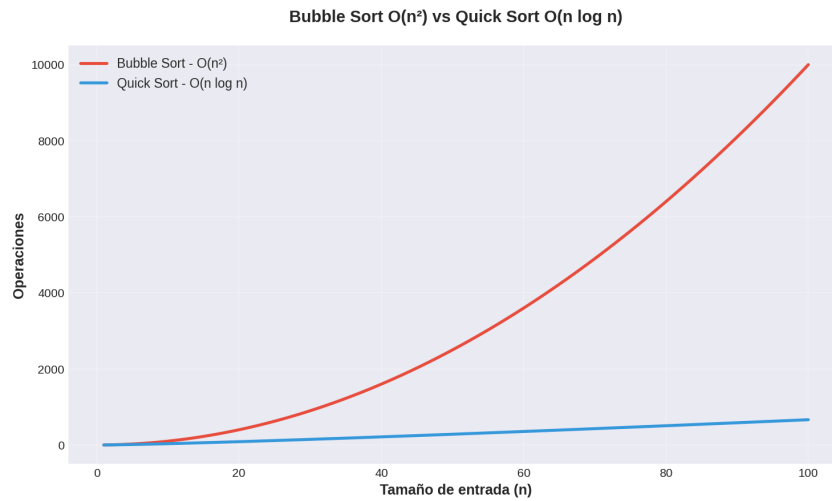
## Ejercicio 7: Bubble Sort vs Quick Sort

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <time.h>
5
6  long long comp_bubble, comp_quick;
7
8  void bubble_sort(int arr[], int n) {
9      for (int i = 0; i < n - 1; i++) {
10         for (int j = 0; j < n - i - 1; j++) {
11             comp_bubble++;
12             if (arr[j] > arr[j + 1]) {
13                 int temp = arr[j];
14                 arr[j] = arr[j + 1];
15                 arr[j + 1] = temp;
16             }
17         }
18     }
19 }
20
21 int partition(int arr[], int low, int high) {
22     int pivot = arr[high];
23     int i = low - 1;
24     for (int j = low; j < high; j++) {
25         comp_quick++;
26         if (arr[j] < pivot) {
27             i++;
28             int temp = arr[i];
29             arr[i] = arr[j];
30             arr[j] = temp;
31         }
32     }
33     int temp = arr[i + 1];
34     arr[i + 1] = arr[high];
35     arr[high] = temp;
36     return i + 1;
37 }
38
39 void quick_sort(int arr[], int low, int high) {
40     if (low < high) {
41         int pi = partition(arr, low, high);
42         quick_sort(arr, low, pi - 1);
43         quick_sort(arr, pi + 1, high);
44     }
45 }
46
47 typedef struct {
48     int n;
49     double tiempo_bubble;
50     double tiempo_quick;
51     long long comp_b;
52     long long comp_q;
53 } ResultadoSort;
54
55 ResultadoSort ejecutar_ordenamiento(int n) {
56     ResultadoSort res;
57     res.n = n;
58
59     int *arr1 = (int*)malloc(n * sizeof(int));
60     int *arr2 = (int*)malloc(n * sizeof(int));
61 }
```

```

62     srand(time(NULL));
63     for (int i = 0; i < n; i++) {
64         arr1[i] = rand() % 10000;
65     }
66     memcpy(arr2, arr1, n * sizeof(int));
67
68     // Bubble Sort
69     comp_bubble = 0;
70     clock_t inicio = clock();
71     bubble_sort(arr1, n);
72     clock_t fin = clock();
73     res.tiempo_bubble = ((double)(fin - inicio)) / CLOCKS_PER_SEC;
74     res.comp_b = comp_bubble;
75
76     // Quick Sort
77     comp_quick = 0;
78     inicio = clock();
79     quick_sort(arr2, 0, n - 1);
80     fin = clock();
81     res.tiempo_quick = ((double)(fin - inicio)) / CLOCKS_PER_SEC;
82     res.comp_q = comp_quick;
83
84     printf("n=%d, Bubble: %.6f seg, Quick: %.6f seg\n",
85           n, res.tiempo_bubble, res.tiempo_quick);
86
87     free(arr1);
88     free(arr2);
89
90     return res;
91 }
92
93 int main() {
94     FILE *archivo;
95     int tamanios[] = {1000, 5000, 10000, 20000, 30000};
96     int num_tamanios = 5;
97
98     archivo = fopen("datos_ejercicio7.csv", "w");
99     fprintf(archivo, "n,tiempo_bubble,tiempo_quick,comp_bubble,comp_quick\n");
100
101     printf("=== EJERCICIO 7: Ordenamiento ===\n\n");
102
103     for (int i = 0; i < num_tamanios; i++) {
104         ResultadoSort r = ejecutar_ordenamiento(tamanios[i]);
105         fprintf(archivo, "%d,%.6f,%.6f,%lld,%lld\n",
106               r.n, r.tiempo_bubble, r.tiempo_quick,
107               r.comp_b, r.comp_q);
108     }
109
110     fclose(archivo);
111
112     return 0;
113 }

```



Captura de compilación:

```
C:\Users\HP\Desktop\ejercici... x + v
=== EJERCICIO 7: Ordenamiento ===

n=1000, Bubble: 0.003000 seg, Quick: 0.001000 seg
n=5000, Bubble: 0.077000 seg, Quick: 0.001000 seg
n=10000, Bubble: 0.306000 seg, Quick: 0.001000 seg
n=20000, Bubble: 1.310000 seg, Quick: 0.000000 seg
n=30000, Bubble: 3.520000 seg, Quick: 0.007000 seg

-----
Process exited after 14.74 seconds with return value 0
Presione una tecla para continuar . . .
```

## Ejercicio 8: Complejidad $O(\sqrt{n})$ - Números Primos

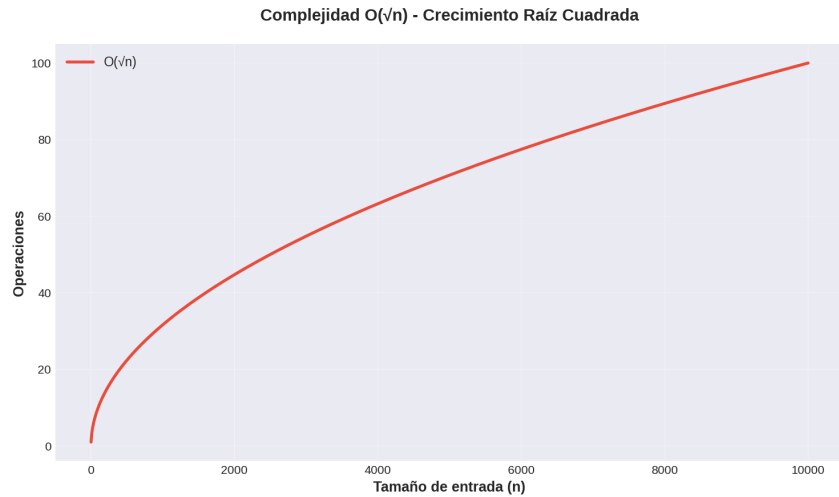
```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4 #include <math.h>
5
6 int es_primo(int n, int *divisiones) {
7     *divisiones = 0;
8     if (n < 2) return 0;
9     if (n == 2) return 1;
10    if (n % 2 == 0) return 0;
11
12    for (int i = 3; i * i <= n; i += 2) {
13        (*divisiones)++;
14        if (n % i == 0) return 0;
15    }
16    return 1;
17 }
18
19 typedef struct {
20     int n;
21     int es_primo;
22     int divisiones;
23     double tiempo;
24     double raiz_n;
25 } ResultadoPrimo;
26
27 ResultadoPrimo ejecutar_primo(int n) {
28     ResultadoPrimo res;
29     res.n = n;
30     res.raiz_n = sqrt(n);
31
32     clock_t inicio = clock();
33     res.es_primo = es_primo(n, &res.divisiones);
34     clock_t fin = clock();
35
36     res.tiempo = ((double)(fin - inicio)) / CLOCKS_PER_SEC;
37
38     printf("n=%d, Primo=%s, Div=%d, sqrt(n)=%.2f, Tiempo=%.6f seg\n",
39         n, res.es_primo ? "SI" : "NO",
40         res.divisiones, res.raiz_n, res.tiempo);
41
42     return res;
43 }
44
45 int main() {
46     FILE *archivo;
47     int numeros[] = {1009, 10007, 100003, 1000003, 10000019,
48         100000007, 1000000007};
49     int num_tests = 7;
50
51     archivo = fopen("datos_ejercicio8.csv", "w");
52     fprintf(archivo, "n,es_primo,divisiones,raiz_n,tiempo_segundos\n");
53
54     printf("=== EJERCICIO 8: Numeros Primos O(sqrt(n)) ===\n\n");
55
56     for (int i = 0; i < num_tests; i++) {
57         ResultadoPrimo r = ejecutar_primo(numeros[i]);
58         fprintf(archivo, "%d,%d,%d,%.2f,%.6f\n",
59             r.n, r.es_primo, r.divisiones, r.raiz_n, r.tiempo);
60     }
61 }
```



```

62     fclose(archivo);
63
64     return 0;
65 }

```



Captura de compilación:

```

C:\Users\HP\Desktop\ejercick x + v
=== EJERCICIO 8: Numeros Primos O(sqrt(n)) ===
n=1009, Primo=SI, Div=15, sqrt(n)=31.76, Tiempo=0.000000 seg
n=10007, Primo=SI, Div=49, sqrt(n)=100.03, Tiempo=0.000000 seg
n=100003, Primo=SI, Div=157, sqrt(n)=316.23, Tiempo=0.000000 seg
n=1000003, Primo=SI, Div=499, sqrt(n)=1000.00, Tiempo=0.000000 seg
n=10000019, Primo=SI, Div=1580, sqrt(n)=3162.28, Tiempo=0.000000 seg
n=100000007, Primo=SI, Div=4999, sqrt(n)=10000.00, Tiempo=0.000000 seg
n=1000000007, Primo=SI, Div=15810, sqrt(n)=31622.78, Tiempo=0.000000 seg

-----
Process exited after 2.523 seconds with return value 0
Presione una tecla para continuar . . .

```

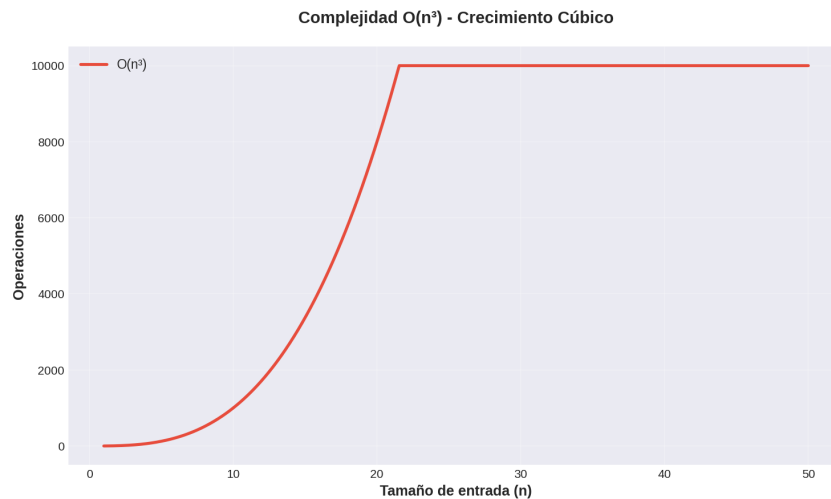
## Ejercicio 9: Complejidad $O(n^3)$ - Multiplicación de Matrices

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 void multiplicar_matrices(int **A, int **B, int **C,
6                           int n, long long *ops) {
7     *ops = 0;
8     for (int i = 0; i < n; i++) {
9         for (int j = 0; j < n; j++) {
10             C[i][j] = 0;
11             for (int k = 0; k < n; k++) {
12                 C[i][j] += A[i][k] * B[k][j];
13                 (*ops)++;
14             }
15         }
16     }
17 }
18
19 double ejecutar_mult_matrices(int n) {
20     long long operaciones = 0;
21
22     int **A = (int**)malloc(n * sizeof(int*));
23     int **B = (int**)malloc(n * sizeof(int*));
24     int **C = (int**)malloc(n * sizeof(int*));
25
26     for (int i = 0; i < n; i++) {
27         A[i] = (int*)malloc(n * sizeof(int));
28         B[i] = (int*)malloc(n * sizeof(int));
29         C[i] = (int*)malloc(n * sizeof(int));
30         for (int j = 0; j < n; j++) {
31             A[i][j] = rand() % 10;
32             B[i][j] = rand() % 10;
33         }
34     }
35
36     clock_t inicio = clock();
37     multiplicar_matrices(A, B, C, n, &operaciones);
38     clock_t fin = clock();
39
40     double tiempo = ((double)(fin - inicio)) / CLOCKS_PER_SEC;
41
42     printf("n=%d, Ops=%lld, Teorico=%lld, Tiempo=%.6f seg\n",
43           n, operaciones, (long long)n * n * n, tiempo);
44
45     // Liberar memoria
46     for (int i = 0; i < n; i++) {
47         free(A[i]);
48         free(B[i]);
49         free(C[i]);
50     }
51     free(A);
52     free(B);
53     free(C);
54
55     return tiempo;
56 }
57
58 int main() {
59     FILE *archivo;
60     int tamanios[] = {50, 100, 200, 300, 500, 800};
61     int num_tamanios = 6;
```

```

62
63     archivo = fopen("datos_ejercicio9.csv", "w");
64     fprintf(archivo, "n,operaciones,tiempo_segundos\n");
65
66     printf("=== EJERCICIO 9: Multiplicacion Matrices O(n^3) ===\n\n");
67
68     for (int i = 0; i < num_tamamos; i++) {
69         int n = tamanios[i];
70         double tiempo = ejecutar_mult_matrices(n);
71         long long ops = (long long)n * n * n;
72         fprintf(archivo, "%d,%lld,%.6f\n", n, ops, tiempo);
73     }
74
75     fclose(archivo);
76
77     return 0;
78 }

```



### Captura de compilación:

```

C:\Users\HP\Desktop\ejercici...
=== EJERCICIO 9: Multiplicacion Matrices O(n^3) ===

n=50, Ops=125000, Teorico=125000, Tiempo=0.002000 seg
n=100, Ops=1000000, Teorico=1000000, Tiempo=0.008000 seg
n=200, Ops=8000000, Teorico=8000000, Tiempo=0.070000 seg
n=300, Ops=27000000, Teorico=27000000, Tiempo=0.234000 seg
n=500, Ops=125000000, Teorico=125000000, Tiempo=1.116000 seg
n=800, Ops=512000000, Teorico=512000000, Tiempo=5.131000 seg

-----
Process exited after 15.8 seconds with return value 0
Presione una tecla para continuar . . .

```

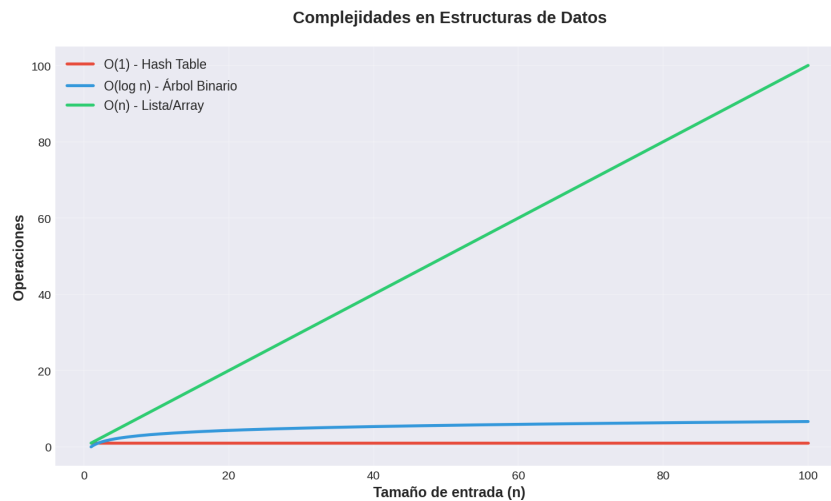
## Ejercicio 10: Comparación de Estructuras de Datos

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  typedef struct Nodo {
6      int dato;
7      struct Nodo *siguiente;
8  } Nodo;
9
10 double medir_acceso_array(int n) {
11     int *arr = (int*)malloc(n * sizeof(int));
12     for (int i = 0; i < n; i++) arr[i] = i;
13
14     clock_t inicio = clock();
15     int suma = 0;
16     for (int i = 0; i < 10000; i++) {
17         suma += arr[n / 2]; // Acceso constante O(1)
18     }
19     clock_t fin = clock();
20
21     free(arr);
22     return ((double)(fin - inicio)) / CLOCKS_PER_SEC;
23 }
24
25 double medir_busqueda_array(int n) {
26     int *arr = (int*)malloc(n * sizeof(int));
27     for (int i = 0; i < n; i++) arr[i] = i;
28
29     clock_t inicio = clock();
30     for (int i = 0; i < 100; i++) {
31         for (int j = 0; j < n; j++) {
32             if (arr[j] == n - 1) break; // O(n)
33         }
34     }
35     clock_t fin = clock();
36
37     free(arr);
38     return ((double)(fin - inicio)) / CLOCKS_PER_SEC;
39 }
40
41 double medir_insercion_lista(int n) {
42     Nodo *lista = NULL;
43
44     clock_t inicio = clock();
45     for (int i = 0; i < n; i++) {
46         Nodo *nuevo = (Nodo*)malloc(sizeof(Nodo));
47         nuevo->dato = i;
48         nuevo->siguiente = lista;
49         lista = nuevo; // O(1)
50     }
51     clock_t fin = clock();
52
53     // Liberar lista
54     while (lista != NULL) {
55         Nodo *temp = lista;
56         lista = lista->siguiente;
57         free(temp);
58     }
59
60     return ((double)(fin - inicio)) / CLOCKS_PER_SEC;
61 }
```

```

62
63 int main() {
64     FILE *archivo;
65     int tamanios[] = {1000, 10000, 50000, 100000, 500000};
66     int num_tamanios = 5;
67
68     archivo = fopen("datos_ejercicio10.csv", "w");
69     fprintf(archivo, "n,tiempo_acceso_array,tiempo_busqueda_array,");
70     fprintf(archivo, "tiempo_insercion_lista\n");
71
72     printf("=== EJERCICIO 10: Estructuras de Datos ===\n\n");
73
74     for (int i = 0; i < num_tamanios; i++) {
75         int n = tamanios[i];
76
77         double t_acceso = medir_acceso_array(n);
78         double t_busqueda = medir_busqueda_array(n);
79         double t_insercion = medir_insercion_lista(n);
80
81         printf("n=%d, Acceso: %.6f, Busqueda: %.6f, Insercion: %.6f\n",
82               n, t_acceso, t_busqueda, t_insercion);
83
84         fprintf(archivo, "%d,%.6f,%.6f,%.6f\n",
85               n, t_acceso, t_busqueda, t_insercion);
86     }
87
88     fclose(archivo);
89     return 0;
90 }

```



### Captura de compilación:

```

C:\Users\HP\Desktop\ejercicio10 > gcc ejercicio10.c -o ejercicio10.exe
=== EJERCICIO 10: Estructuras de Datos ===
n=1000, Acceso: 0.000000, Busqueda: 0.000000, Insercion: 0.000000
n=10000, Acceso: 0.006000, Busqueda: 0.003000, Insercion: 0.001000
n=50000, Acceso: 0.000000, Busqueda: 0.022000, Insercion: 0.003000
n=100000, Acceso: 0.000000, Busqueda: 0.041000, Insercion: 0.010000
n=500000, Acceso: 0.000000, Busqueda: 0.212000, Insercion: 0.041000
-----
Process exited after 4.434 seconds with return value 0
Presione una tecla para continuar . . .

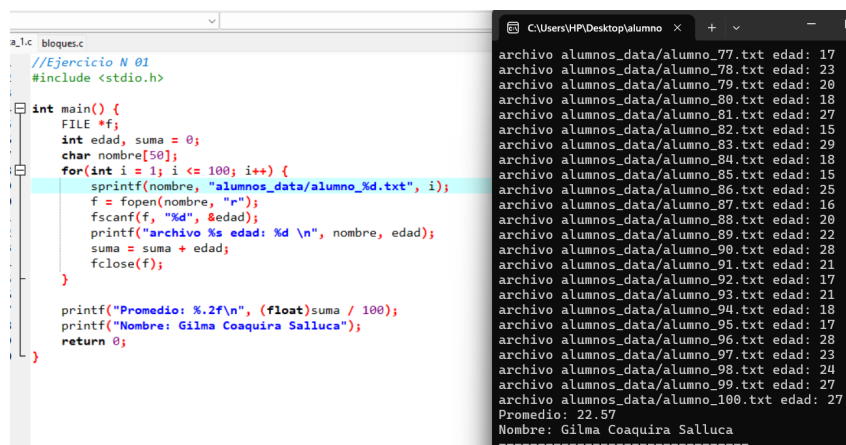
```

## PARTE 2: CÓDIGOS BASE - MANEJO DE ARCHIVOS

### Código 1: Promedio de edades

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *f;
5     int edad, suma = 0;
6     char nombre[50];
7
8     for(int i = 1; i <= 100; i++) {
9         sprintf(nombre, "alumnos_data/alumno_%d.txt", i);
10        f = fopen(nombre, "r");
11        if (f == NULL) {
12            printf("Error: %s\n", nombre);
13            continue;
14        }
15        fscanf(f, "%d", &edad);
16        printf("archivo %s edad: %d\n", nombre, edad);
17        suma = suma + edad;
18        fclose(f);
19    }
20
21    printf("\nPromedio: %.2f\n", (float)suma / 100);
22    return 0;
23 }
```

### Captura de compilación:



```
a1.c bloques.c
//Ejercicio N 01
#include <stdio.h>

int main() {
    FILE *f;
    int edad, suma = 0;
    char nombre[50];
    for(int i = 1; i <= 100; i++) {
        sprintf(nombre, "alumnos_data/alumno_%d.txt", i);
        f = fopen(nombre, "r");
        fscanf(f, "%d", &edad);
        printf("archivo %s edad: %d\n", nombre, edad);
        suma = suma + edad;
        fclose(f);
    }

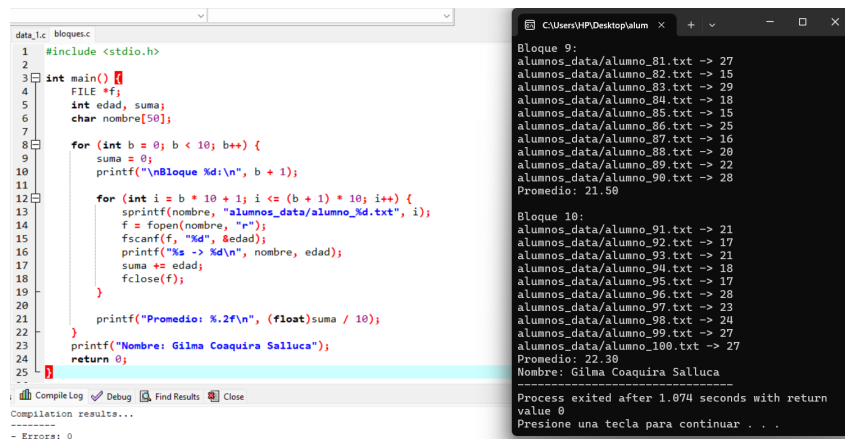
    printf("Promedio: %.2f\n", (float)suma / 100);
    printf("Nombre: Gilma Coaquira Salluca");
    return 0;
}
```

```
C:\Users\HP\Desktop\alumno x + v - □
archivo alumnos_data/alumno_77.txt edad: 17
archivo alumnos_data/alumno_78.txt edad: 23
archivo alumnos_data/alumno_79.txt edad: 20
archivo alumnos_data/alumno_80.txt edad: 18
archivo alumnos_data/alumno_81.txt edad: 27
archivo alumnos_data/alumno_82.txt edad: 15
archivo alumnos_data/alumno_83.txt edad: 29
archivo alumnos_data/alumno_84.txt edad: 18
archivo alumnos_data/alumno_85.txt edad: 15
archivo alumnos_data/alumno_86.txt edad: 25
archivo alumnos_data/alumno_87.txt edad: 16
archivo alumnos_data/alumno_88.txt edad: 20
archivo alumnos_data/alumno_89.txt edad: 22
archivo alumnos_data/alumno_90.txt edad: 28
archivo alumnos_data/alumno_91.txt edad: 21
archivo alumnos_data/alumno_92.txt edad: 17
archivo alumnos_data/alumno_93.txt edad: 21
archivo alumnos_data/alumno_94.txt edad: 18
archivo alumnos_data/alumno_95.txt edad: 17
archivo alumnos_data/alumno_96.txt edad: 28
archivo alumnos_data/alumno_97.txt edad: 23
archivo alumnos_data/alumno_98.txt edad: 24
archivo alumnos_data/alumno_99.txt edad: 27
archivo alumnos_data/alumno_100.txt edad: 27
Promedio: 22.57
Nombre: Gilma Coaquira Salluca
```

## Código 2: Bloques de 10 y promedios de cada bloque

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *f;
5     int edad, suma;
6     char nombre[50];
7
8     for (int b = 0; b < 10; b++) {
9         suma = 0;
10        printf("\nBloque %d:\n", b + 1);
11
12        for (int i = b * 10 + 1; i <= (b + 1) * 10; i++) {
13            sprintf(nombre, "alumnos_data/alumno_%d.txt", i);
14            f = fopen(nombre, "r");
15            if (f == NULL) {
16                printf("Error: %s\n", nombre);
17                continue;
18            }
19            fscanf(f, "%d", &edad);
20            printf("%s -> %d\n", nombre, edad);
21            suma += edad;
22            fclose(f);
23        }
24
25        printf("Promedio: %.2f\n", (float)suma / 10);
26    }
27    return 0;
28 }
```

### Captura de compilación:



The image shows a code editor window on the left and a terminal window on the right. The code editor displays the C program from the previous block, with line numbers 1 through 28. The terminal window shows the output of the program, which is divided into two sections: 'Bloque 9:' and 'Bloque 10:'. Each section lists 10 files (alumno\_81.txt to alumno\_90.txt for Bloque 9, and alumno\_91.txt to alumno\_100.txt for Bloque 10) and their corresponding ages. The average for Bloque 9 is 21.50, and for Bloque 10 is 22.30. The program ends with the name 'Gilma Coaquira Salluca' and a message 'Process exited after 1.074 seconds with return value 0'. The terminal also shows a prompt 'Presione una tecla para continuar...'.

```
data1.c bloques.c
1 #include <stdio.h>
2
3 int main() {
4     FILE *f;
5     int edad, suma;
6     char nombre[50];
7
8     for (int b = 0; b < 10; b++) {
9         suma = 0;
10        printf("\nBloque %d:\n", b + 1);
11
12        for (int i = b * 10 + 1; i <= (b + 1) * 10; i++) {
13            sprintf(nombre, "alumnos_data/alumno_%d.txt", i);
14            f = fopen(nombre, "r");
15            if (f == NULL) {
16                printf("Error: %s\n", nombre);
17                continue;
18            }
19            fscanf(f, "%d", &edad);
20            printf("%s -> %d\n", nombre, edad);
21            suma += edad;
22            fclose(f);
23        }
24
25        printf("Promedio: %.2f\n", (float)suma / 10);
26    }
27    return 0;
28 }
```

```
C:\Users\HP\Desktop\alum x + - _
Bloque 9:
alumnos_data/alumno_81.txt -> 27
alumnos_data/alumno_82.txt -> 15
alumnos_data/alumno_83.txt -> 29
alumnos_data/alumno_84.txt -> 18
alumnos_data/alumno_85.txt -> 15
alumnos_data/alumno_86.txt -> 25
alumnos_data/alumno_87.txt -> 16
alumnos_data/alumno_88.txt -> 28
alumnos_data/alumno_89.txt -> 22
alumnos_data/alumno_90.txt -> 28
Promedio: 21.50

Bloque 10:
alumnos_data/alumno_91.txt -> 21
alumnos_data/alumno_92.txt -> 17
alumnos_data/alumno_93.txt -> 21
alumnos_data/alumno_94.txt -> 18
alumnos_data/alumno_95.txt -> 17
alumnos_data/alumno_96.txt -> 28
alumnos_data/alumno_97.txt -> 23
alumnos_data/alumno_98.txt -> 24
alumnos_data/alumno_99.txt -> 27
alumnos_data/alumno_100.txt -> 27
Promedio: 22.30
Nombre: Gilma Coaquira Salluca
-----
Process exited after 1.074 seconds with return
value 0
Presione una tecla para continuar . . .
```

## PARTE 3: EJERCICIOS DE PRÁCTICA

### Ejercicio Práctica 1: Sistema de Registro de Temperaturas

```
1  #include <stdio.h>
2
3  int main() {
4      FILE *archivo;
5      float temp, suma = 0, max = -999, min = 999;
6      int n = 5;
7
8      archivo = fopen("temperaturas.txt", "w");
9      fprintf(archivo, "MEDICIONES:\n");
10
11     printf("Ingrese 5 temperaturas (grados Celsius):\n");
12     for (int i = 0; i < n; i++) {
13         printf("Temperatura %d: ", i+1);
14         scanf("%f", &temp);
15         fprintf(archivo, "%.1f\n", temp);
16     }
17     fclose(archivo);
18
19     archivo = fopen("temperaturas.txt", "r");
20     fscanf(archivo, "%*[^\\n]\\n");
21
22     while (fscanf(archivo, "%f", &temp) == 1) {
23         suma += temp;
24         if (temp > max) max = temp;
25         if (temp < min) min = temp;
26     }
27     fclose(archivo);
28
29     float promedio = suma / n;
30
31     archivo = fopen("temperaturas.txt", "a");
32     fprintf(archivo, "ESTADISTICAS:\n");
33     fprintf(archivo, "Promedio: %.2f\n", promedio);
34     fprintf(archivo, "Maxima: %.2f\n", max);
35     fprintf(archivo, "Minima: %.2f\n", min);
36     fclose(archivo);
37
38     printf("\\n=== ESTADISTICAS ===\\n");
39     printf("Promedio: %.2f C\\n", promedio);
40     printf("Maxima: %.2f C\\n", max);
41     printf("Minima: %.2f C\\n", min);
42     printf("\\nArchivo: temperaturas.txt\\n");
43
44     return 0;
45 }
```

Captura de compilación:



```
C:\Users\HP\Desktop\ejercicio x + v
Ingrese 5 temperaturas (grados Celsius):
Temperatura 1: 5
Temperatura 2: 3
Temperatura 3: 2
Temperatura 4: 5
Temperatura 5: 2

=== ESTADISTICAS ===
Promedio: 3.40 C
Maxima: 5.00 C
Minima: 2.00 C

Archivo: temperaturas.txt

-----
Process exited after 15.9 seconds with return value 0
Presione una tecla para continuar . . .
```

## Ejercicio Práctica 2: Base de Datos Simple de Productos

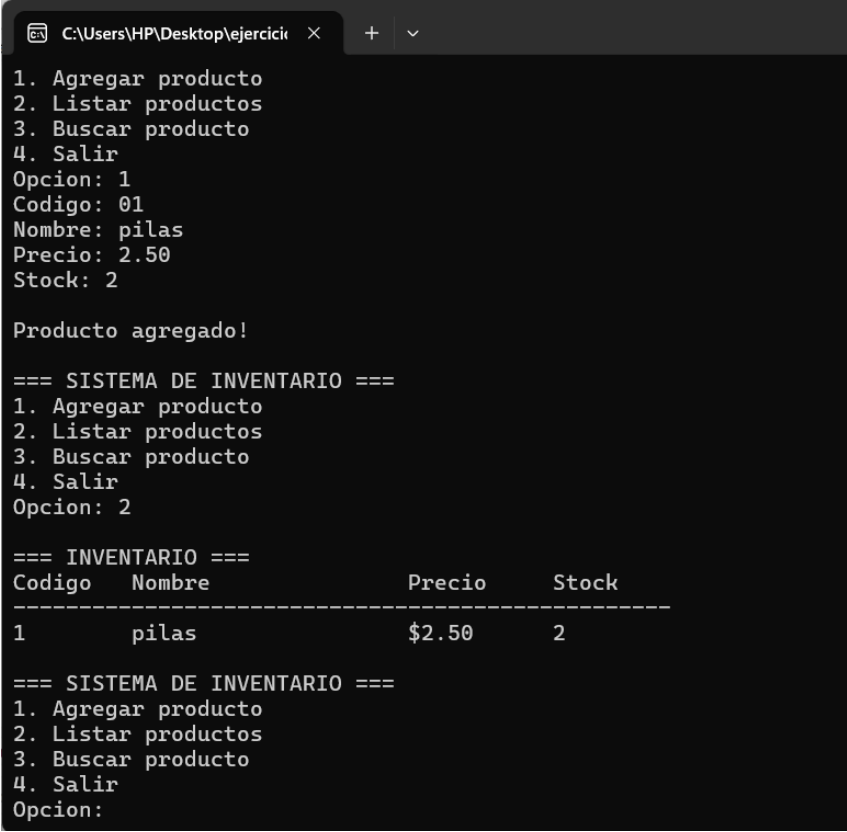
```
1  #include <stdio.h>
2  #include <string.h>
3
4  typedef struct {
5      int codigo;
6      char nombre[30];
7      float precio;
8      int stock;
9  } Producto;
10
11 void agregar_producto() {
12     FILE *archivo = fopen("inventario.dat", "ab");
13     Producto p;
14     printf("Codigo: "); scanf("%d", &p.codigo);
15     printf("Nombre: "); scanf("%[^\n]", p.nombre);
16     printf("Precio: "); scanf("%f", &p.precio);
17     printf("Stock: "); scanf("%d", &p.stock);
18     fwrite(&p, sizeof(Producto), 1, archivo);
19     fclose(archivo);
20     printf("\nProducto agregado!\n");
21 }
22
23 void listar_productos() {
24     FILE *archivo = fopen("inventario.dat", "rb");
25     if (archivo == NULL) {
26         printf("No hay productos\n");
27         return;
28     }
29     Producto p;
30     printf("\n=== INVENTARIO ===\n");
31     printf("%-8s %-20s %-10s %s\n", "Codigo", "Nombre",
32         "Precio", "Stock");
33     printf("-----\n");
34     while (fread(&p, sizeof(Producto), 1, archivo)) {
35         printf("%-8d %-20s $%-9.2f %d\n",
36             p.codigo, p.nombre, p.precio, p.stock);
37     }
38     fclose(archivo);
39 }
40
41 void buscar_producto() {
42     int codigo;
43     printf("Codigo a buscar: "); scanf("%d", &codigo);
44     FILE *archivo = fopen("inventario.dat", "rb");
45     if (archivo == NULL) {
46         printf("No hay productos\n");
47         return;
48     }
49     Producto p;
50     int encontrado = 0;
51     while (fread(&p, sizeof(Producto), 1, archivo)) {
52         if (p.codigo == codigo) {
53             printf("\nProducto encontrado:\n");
54             printf("Nombre: %s\n", p.nombre);
55             printf("Precio: $%.2f\n", p.precio);
56             printf("Stock: %d\n", p.stock);
57             encontrado = 1;
58             break;
59         }
60     }
61     if (!encontrado) printf("Producto no encontrado\n");
```

```

62     fclose(archivo);
63 }
64
65 int main() {
66     int opcion;
67     do {
68         printf("\n=== SISTEMA DE INVENTARIO ===\n");
69         printf("1. Agregar producto\n");
70         printf("2. Listar productos\n");
71         printf("3. Buscar producto\n");
72         printf("4. Salir\n");
73         printf("Opcion: "); scanf("%d", &opcion);
74
75         switch(opcion) {
76             case 1: agregar_producto(); break;
77             case 2: listar_productos(); break;
78             case 3: buscar_producto(); break;
79             default: printf("Opcion invalida\n");
80         }
81     } while(opcion != 4);
82     return 0;
83 }

```

### Captura de compilación:



```

C:\Users\HP\Desktop\ejercicio >
1. Agregar producto
2. Listar productos
3. Buscar producto
4. Salir
Opcion: 1
Codigo: 01
Nombre: pilas
Precio: 2.50
Stock: 2

Producto agregado!

=== SISTEMA DE INVENTARIO ===
1. Agregar producto
2. Listar productos
3. Buscar producto
4. Salir
Opcion: 2

=== INVENTARIO ===
Codigo  Nombre          Precio  Stock
-----
1      pilas              $2.50   2

=== SISTEMA DE INVENTARIO ===
1. Agregar producto
2. Listar productos
3. Buscar producto
4. Salir
Opcion:

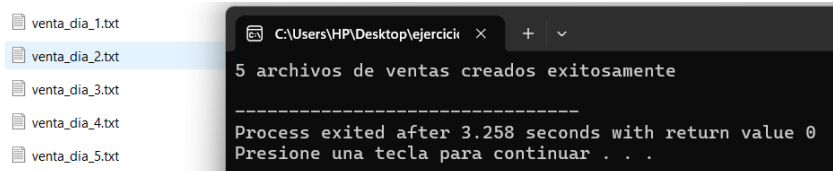
```

## PARTE 4: EJERCICIOS DE MANEJO DE ARCHIVOS

### Ejercicio Archivos 1: Registro de Ventas Diarias

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo;
5     char nombre[30];
6     float ventas[] = {1500.50, 2300.75, 1800.00, 2100.25, 1950.80};
7
8     for (int dia = 1; dia <= 5; dia++) {
9         sprintf(nombre, "venta_dia_%d.txt", dia);
10        archivo = fopen(nombre, "w");
11        fprintf(archivo, "%.2f\n", ventas[dia-1]);
12        fclose(archivo);
13    }
14
15    printf("5 archivos de ventas creados exitosamente\n");
16    return 0;
17 }
```

Salida de compilación:



### Ejercicio Archivos 2: Calcular Total de Ventas

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo;
5     char nombre[30];
6     float venta, total = 0;
7
8     for (int dia = 1; dia <= 5; dia++) {
9         sprintf(nombre, "venta_dia_%d.txt", dia);
10        archivo = fopen(nombre, "r");
11        fscanf(archivo, "%f", &venta);
12        printf("Dia %d: $%.2f\n", dia, venta);
13        total += venta;
14        fclose(archivo);
15    }
16
17    printf("\nTotal de ventas: $%.2f\n", total);
18    return 0;
19 }
```

Salida de compilación:

```
C:\Users\HP\Desktop\ejercicio >
Dia 1: $1500.50
Dia 2: $2300.75
Dia 3: $1800.00
Dia 4: $2100.25
Dia 5: $1950.80

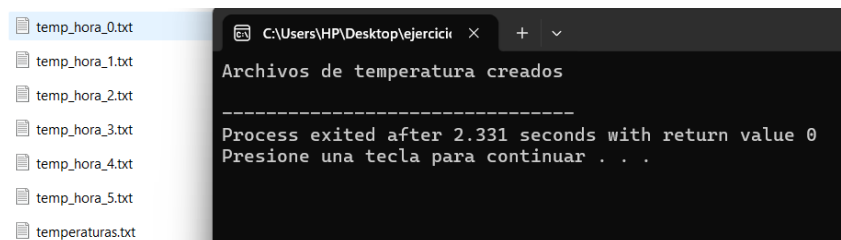
Total de ventas: $9652.30

-----
Process exited after 3.534 seconds with return value 0
Presione una tecla para continuar . . .
```

## Ejercicio Archivos 3: Registro de Temperaturas por Hora

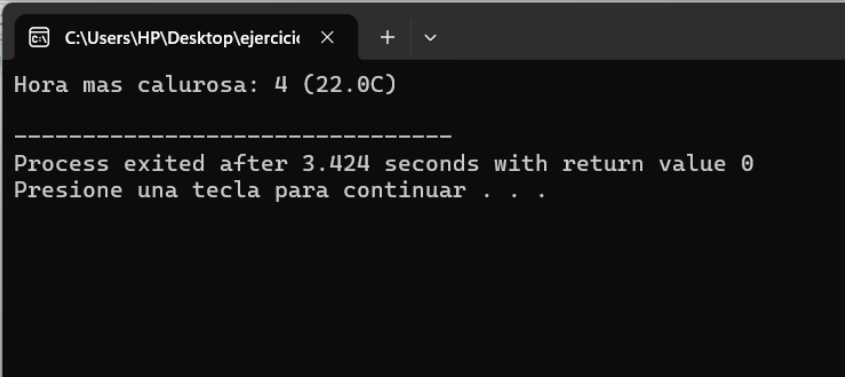
```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo;
5     char nombre[30];
6     float temperaturas[] = {18.5, 19.2, 20.1, 21.5, 22.0, 21.8};
7
8     for (int hora = 0; hora < 6; hora++) {
9         sprintf(nombre, "temp_hora_%d.txt", hora);
10        archivo = fopen(nombre, "w");
11        fprintf(archivo, "Hora %d: %.1fC\n", hora, temperaturas[hora]);
12        fclose(archivo);
13    }
14
15    printf("Archivos de temperatura creados\n");
16    return 0;
17 }
```

Salida de compilación:



## Ejercicio Archivos 4: Encontrar Temperatura Máxima

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo;
5     char nombre[30];
6     int hora_actual;
7     float temp, temp_max = -999;
8     int hora_max = 0;
9
10    for (int h = 0; h < 6; h++) {
11        sprintf(nombre, "temp_hora_%d.txt", h);
12        archivo = fopen(nombre, "r");
13        fscanf(archivo, "Hora %d: %fC", &hora_actual, &temp);
14
15        if (temp > temp_max) {
16            temp_max = temp;
17            hora_max = hora_actual;
18        }
19        fclose(archivo);
20    }
21
22    printf("Hora mas calurosa: %d (%.1fC)\n", hora_max, temp_max);
23    return 0;
24 }
```

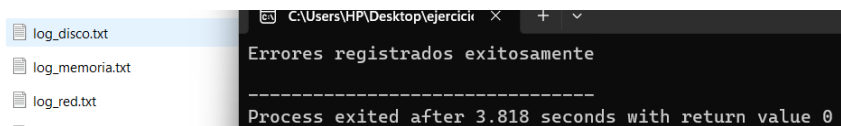
**Salida de compilación:**A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\HP\Desktop\ejercici" and standard window controls. The command prompt displays the output of a program: "Hora mas calurosa: 4 (22.0C)", followed by a separator line of dashes, and then "Process exited after 3.424 seconds with return value 0" and "Presione una tecla para continuar . . .".

```
C:\Users\HP\Desktop\ejercici >
Hora mas calurosa: 4 (22.0C)
-----
Process exited after 3.424 seconds with return value 0
Presione una tecla para continuar . . .
```

## Ejercicio Archivos 5: Sistema de Logs de Errores

```
1 #include <stdio.h>
2
3 void registrar_error(char *tipo, char *mensaje) {
4     FILE *archivo;
5     char nombre[40];
6     sprintf(nombre, "log_%s.txt", tipo);
7     archivo = fopen(nombre, "a");
8     fprintf(archivo, "%s\n", mensaje);
9     fclose(archivo);
10 }
11
12 int main() {
13     registrar_error("red", "Conexion perdida");
14     registrar_error("disco", "Espacio insuficiente");
15     registrar_error("red", "Timeout en servidor");
16     registrar_error("memoria", "Out of memory");
17
18     printf("Errores registrados exitosamente\n");
19     return 0;
20 }
```

Salida de compilación:



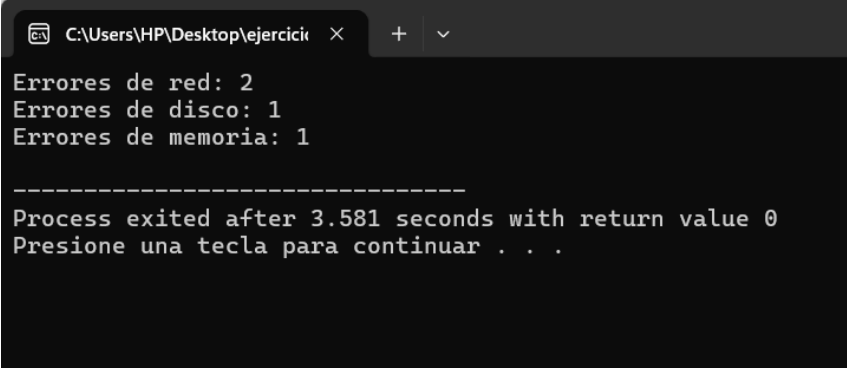
## Ejercicio Archivos 6: Contar Líneas en Log

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo;
5     char linea[100];
6     char tipos[3][20] = {"red", "disco", "memoria"};
7
8     for (int i = 0; i < 3; i++) {
9         char nombre[40];
10        sprintf(nombre, "log_%s.txt", tipos[i]);
11        archivo = fopen(nombre, "r");
12
13        if (archivo == NULL) {
14            printf("Log %s: 0 errores\n", tipos[i]);
15            continue;
16        }
17
18        int contador = 0;
19        while (fgets(linea, 100, archivo) != NULL) {
20            contador++;
21        }
22        fclose(archivo);
23
24        printf("Errores de %s: %d\n", tipos[i], contador);
25    }
```



```
25     }  
26  
27     return 0;  
28 }
```

Salida de compilación:

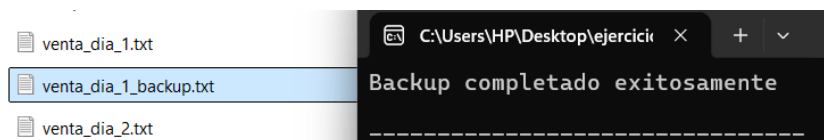


```
C:\Users\HP\Desktop\ejercicio > .\ejercicio.exe  
Errores de red: 2  
Errores de disco: 1  
Errores de memoria: 1  
  
-----  
Process exited after 3.581 seconds with return value 0  
Presione una tecla para continuar . . .
```

## Ejercicio Archivos 7: Backup de Archivos

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *origen, *destino;
5     char c;
6
7     origen = fopen("venta_dia_1.txt", "r");
8     destino = fopen("venta_dia_1_backup.txt", "w");
9
10    while ((c = fgetc(origen)) != EOF) {
11        fputc(c, destino);
12    }
13
14    fclose(origen);
15    fclose(destino);
16
17    printf("Backup completado exitosamente\n");
18    return 0;
19 }
```

Salida de compilación:

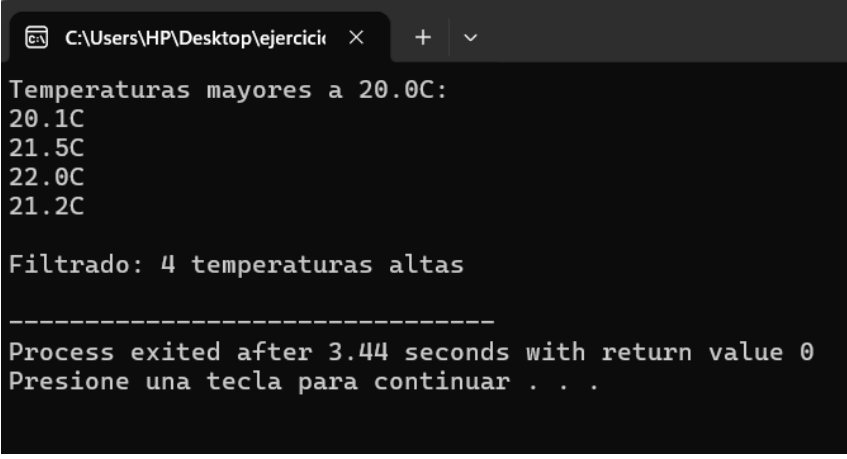


## Ejercicio Archivos 8: Filtrar Temperaturas Altas

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *entrada, *salida;
5     float temp;
6     int contador = 0;
7
8     entrada = fopen("todas_temps.txt", "w");
9     float temps[] = {18.5, 19.2, 20.1, 21.5, 22.0, 19.8, 21.2};
10    for (int i = 0; i < 7; i++) {
11        fprintf(entrada, "%.1f\n", temps[i]);
12    }
13    fclose(entrada);
14
15    entrada = fopen("todas_temps.txt", "r");
16    salida = fopen("temps_altas.txt", "w");
17
18    printf("Temperaturas mayores a 20.0C:\n");
19    while (fscanf(entrada, "%f", &temp) == 1) {
20        if (temp > 20.0) {
21            fprintf(salida, "%.1f\n", temp);
22            printf("%.1fC\n", temp);
23            contador++;
24        }
25    }
```

```
25     }
26
27     fclose(entrada);
28     fclose(salida);
29
30     printf("\nFiltrado: %d temperaturas altas\n", contador);
31     return 0;
32 }
```

Salida de compilación:



```
C:\Users\HP\Desktop\ejercicio >
Temperaturas mayores a 20.0C:
20.1C
21.5C
22.0C
21.2C

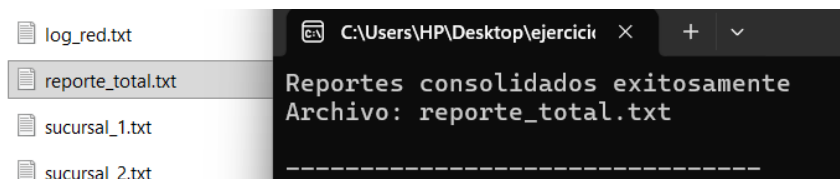
Filtrado: 4 temperaturas altas

-----
Process exited after 3.44 seconds with return value 0
Presione una tecla para continuar . . .
```

## Ejercicio Archivos 9: Consolidar Reportes

```
1  #include <stdio.h>
2
3  int main() {
4      FILE *entrada, *salida;
5      char nombre[30], linea[100];
6
7      // Crear archivos de prueba
8      for (int i = 1; i <= 3; i++) {
9          sprintf(nombre, "sucursal_%d.txt", i);
10         entrada = fopen(nombre, "w");
11         fprintf(entrada, "Sucursal %d\n", i);
12         fprintf(entrada, "Ventas: $%.2f\n", 1000.0 * i);
13         fprintf(entrada, "----\n");
14         fclose(entrada);
15     }
16
17     // Consolidar
18     salida = fopen("reporte_total.txt", "w");
19     fprintf(salida, "=== REPORTE CONSOLIDADO ===\n\n");
20
21     for (int i = 1; i <= 3; i++) {
22         sprintf(nombre, "sucursal_%d.txt", i);
23         entrada = fopen(nombre, "r");
24
25         while (fgets(linea, 100, entrada) != NULL) {
26             fputs(linea, salida);
27         }
28         fclose(entrada);
29     }
30
31     fclose(salida);
32
33     printf("Reportes consolidados exitosamente\n");
34     printf("Archivo: reporte_total.txt\n");
35     return 0;
36 }
```

Salida de compilación:



## Ejercicio Archivos 10: Procesar CSV de Calificaciones

```
1  #include <stdio.h>
2
3  int main() {
4      FILE *archivo;
5      char nombre[50];
6      float nota1, nota2, nota3, promedio;
7      float suma_promedios = 0;
8      int contador = 0;
9
10     // Crear archivo CSV
11     archivo = fopen("notas.csv", "w");
12     fprintf(archivo, "Nombre,Nota1,Nota2,Nota3\n");
13     fprintf(archivo, "Ana Martinez,18.5,17.0,19.0\n");
14     fprintf(archivo, "Carlos Lopez,16.5,15.0,17.5\n");
15     fprintf(archivo, "Maria Silva,19.0,18.5,19.5\n");
16     fprintf(archivo, "Pedro Gomez,15.0,16.0,14.5\n");
17     fclose(archivo);
18
19     // Procesar CSV
20     archivo = fopen("notas.csv", "r");
21     fscanf(archivo, "%*[^\\n]\\n");
22
23     printf("=== CALIFICACIONES ===\\n");
24     printf("%-20s %6s %6s %6s %8s\\n",
25           "Nombre", "Nota1", "Nota2", "Nota3", "Promedio");
26     printf("-----\\n");
27
28     while (fscanf(archivo, "%[^,],%f,%f,%f\\n",
29           nombre, &nota1, &nota2, &nota3) == 4) {
30         promedio = (nota1 + nota2 + nota3) / 3;
31         printf("%-20s %6.1f %6.1f %6.1f %8.2f\\n",
32               nombre, nota1, nota2, nota3, promedio);
33         suma_promedios += promedio;
34         contador++;
35     }
36
37     fclose(archivo);
38
39     printf("-----\\n");
40     printf("Promedio general: %.2f\\n", suma_promedios / contador);
41     printf("Total estudiantes: %d\\n", contador);
42     return 0;
43 }
```

Salida de compilación:

```
C:\Users\HP\Desktop\ejercici x + v
=== CALIFICACIONES ===
Nombre          Nota1  Nota2  Nota3 Promedio
-----
Ana Martinez    18.5   17.0   19.0   18.17
Carlos Lopez    16.5   15.0   17.5   16.33
Maria Silva     19.0   18.5   19.5   19.00
Pedro Gomez     15.0   16.0   14.5   15.17
-----
Promedio general: 17.17
Total estudiantes: 4

-----
Process exited after 3.865 seconds with return value 0
Presione una tecla para continuar . . .
```